

8-bit Bit-Serial Adder

One full adder, eight clock cycles, datapath + FSM

Bibek Shrestha | September 2026 | SystemVerilog, VCS, Design Compiler, SAED 90 nm

The assignment

Design a unit that adds two 8-bit operands using a **single full adder** (bit-serial). One 8-bit bus provides A and B; a separate 8-bit bus outputs the result. 1dA stores A, 1dB stores B and starts the addition on the next clock. A carry register can be cleared or set from outside and is visible on CarryOut. RDY goes high when the result (in register A) is ready. Split datapath and control, write a testbench, synthesize and report as in Assignment 1.

At a glance

Architecture	datapath + 3-state FSM (IDLE, ADD, DONE)
Flip-flops	23
Cell area / total area	1268.12 / 1332.54
Clock target	5 ns
Slack	+1.77 ns MET
Critical path	input 1dB → control → bit_count_reg[3]
Latency	9 clocks from 1dB to RDY
RTL and gate-level tests	all PASS

1 How it works

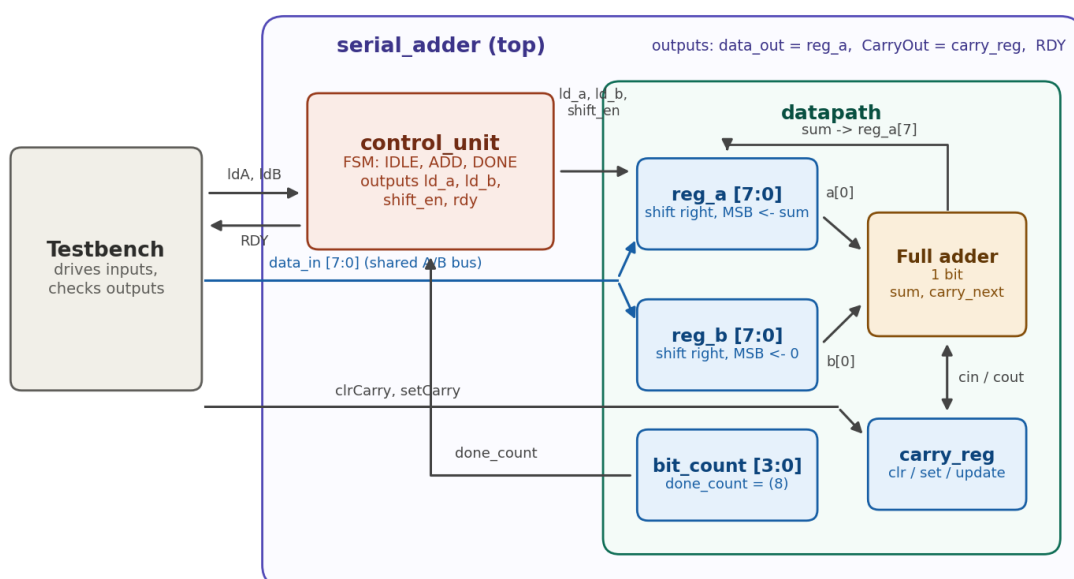
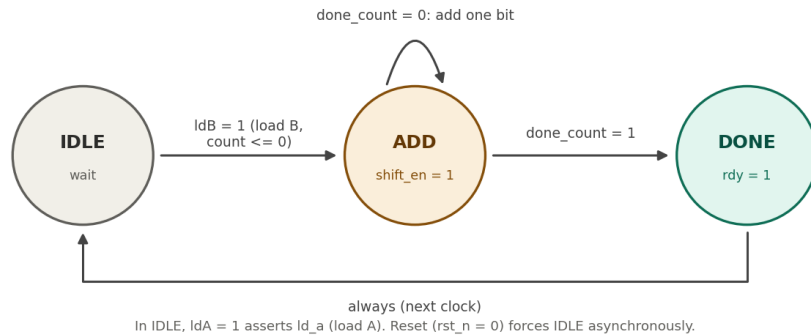


Figure 1: Block diagram. The control unit tells the datapath what to do; the datapath reports back when 8 bits are done.

Part	Bits	Job
reg_a	8	Holds A. Shifts right each step; the new sum bit enters at the top. Ends up holding the result.
reg_b	8	Holds B. Shifts right, presenting the next bit at reg_b[0].
Full adder	1	$\text{sum} = a0 \oplus b0 \oplus c$, $\text{carry} = a0 \& b0 \mid a0 \& c \mid b0 \& c$
carry_reg	1	Carries between bits. clrCarry clears it, setCarry sets it (carry-in).
bit_count	4	Counts shifts; done_count = (bit_count == 8).

Table 1: Datapath parts.**Figure 2:** Control unit. It waits in IDLE, adds one bit per clock in ADD, and pulses RDY in DONE.

Synchronous or asynchronous? Everything changes on the rising clock edge, except the active-low reset `rst_n`, which is asynchronous (it is in the `always_ff` sensitivity list).

2 Example: 3 + 5, one bit per clock

Shift	a0	b0	carry in	sum	carry out	reg_a after	reg_b after
1	1	1	0	0	1	00000001	00000010
2	1	0	1	0	1	00000000	00000001
3	0	1	1	0	1	00000000	00000000
4	0	0	1	1	0	10000000	00000000
5 to 8	0	0	0	0	0	... → 00001000	00000000

Table 2: After 8 shifts register A holds 8.

The DVE waveform shows exactly this on `data_out`: 03, 01, 00, 80, 40, 20, 10, 08.

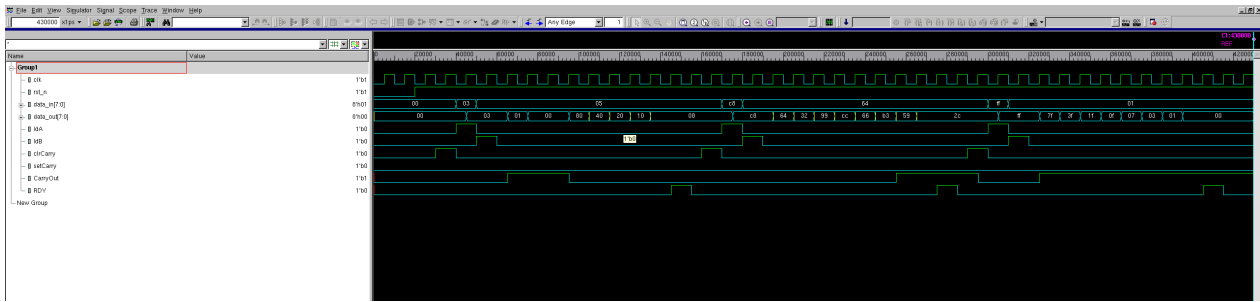


Figure 3: Simulation of $3+5$, $200+100$ and $255+1$: RDY pulses once per addition and CarryOut goes high for the two overflow cases.

Latency: 8 shift cycles + 1 cycle to see the counter reach 8 = **9 clocks** from `ldB` to RDY.

3 Testbench and results

Stimulus changes on the *falling* edge, so there is no race with the rising edge (the problem seen in Assignment 1). Two tasks do a full transaction: `do_add` clears the carry first, `do_add_cin` sets it first.

Test	Expected	RTL	Gate level	
$3 + 5$	8, carry 0	8, 0	8, 0	PASS
$200 + 100$	44, carry 1 (300 wraps)	44, 1	44, 1	PASS
$255 + 1$	0, carry 1 (carry through all bits)	0, 1	0, 1	PASS
$3 + 5 + \text{carry-in}$	9, carry 0		9, 0	PASS
$255 + 0 + \text{carry-in}$	0, carry 1		0, 1	PASS

Table 3: Test results. The carry-in tests were added later to cover `setCarry` and were run on the synthesized netlist.

4 Synthesis

Same flow as Assignment 1, using `adder_flow.tcl`: 5 ns clock, 2 ns input/output delay, `set_max_area 0`, netlist written with `-hierarchy` so both submodules are kept.

```
dc_shell -f adder_flow.tcl
```

Registers		Area	
state	2	Combinational	526.23
bit_count	4	Flip-flops	741.89
reg_a	8	Nets	64.41
reg_b	8	Cell area	1268.12
carry_reg	1	Total area	1332.54
Total	23	Cells (comb. / seq.)	51 / 23

Table 4: Registers found at elaboration and the area report. Flip-flops are 58% of the cell area.

Critical path

input external delay	2.00	2.00	
ldB (in)	0.00	2.00	
u_ctrl/U12/Q (AND2X1)	0.13	2.13	ld_b = ldB in IDLE
u_dp/U3, U4 (NOR2X0)	0.46	2.58	
u_dp/U41, U38, U35 (A021X1)	0.41	2.99	bit counter logic
u_dp/U34/Q (A022X1)	0.12	3.11	
u_dp/bit_count_reg[3]/D (DFFARX1)	0.03	3.14	
data required time (5.00 - 0.08)		4.92	
slack (MET)		1.77	

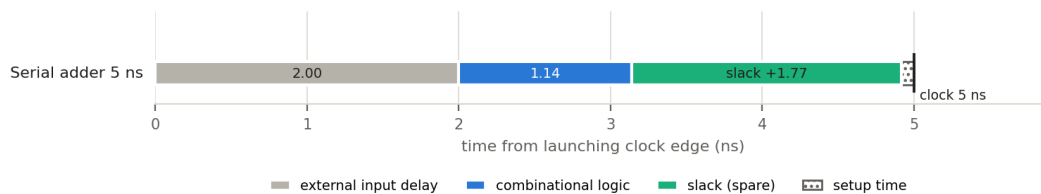


Figure 4: Timing budget. 2.00 ns is the assumed outside delay; only 1.14 ns is logic inside the design.

The adder is not the critical path

A single full adder is only a couple of gates. The slowest path is control: the ldB input going through the FSM and into the bit counter. That is the upside of a serial design: very little logic per clock.

5 Design Vision

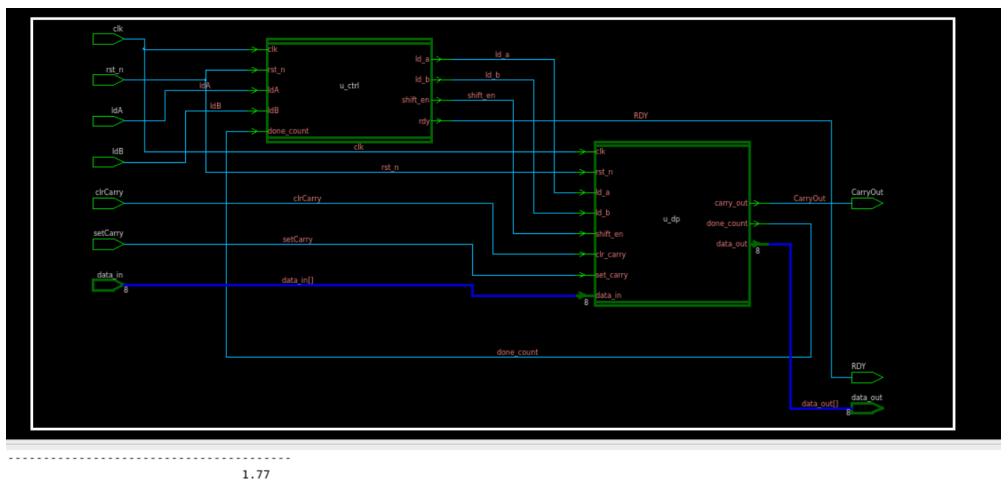


Figure 5: Synthesized top level: one 8-bit data_in bus in, one 8-bit data_out bus out, u_ctrl and u_dp connected as designed.

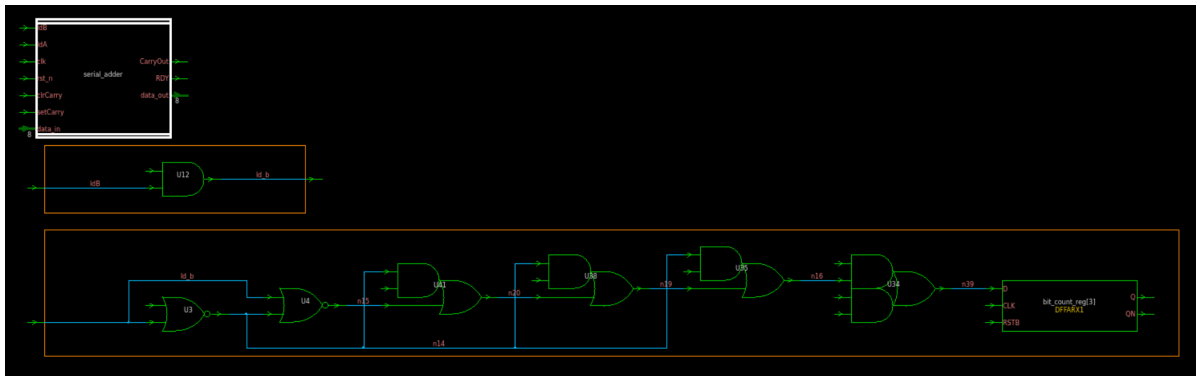


Figure 6: The critical path: 1dB → AND gate in the control unit → bit counter logic → bit_count_reg[3].

6 Requirements check

Requirement	How	
Two 8-bit operands, single full adder, bit-serial	reg_a, reg_b, 1-bit adder, 8 shifts	PASS
One shared 8-bit input bus for A and B	data_in[7:0]	PASS
Separate 8-bit output bus with result from register A	data_out = reg_a	PASS
1dA loads A; 1dB loads B and starts next clock	FSM IDLE → ADD	PASS
Carry clear/set from outside, visible on CarryOut	clrCarry, setCarry, carry_reg	PASS
RDY when result is ready	DONE state	PASS
Datapath and control unit split	datapath, control_unit	PASS
Synthesis and reports as in Assignment 1	baseline done; speed and area runs to do	partial

Still to do: run the speed (3 ns, high/low) and area (5 ns, medium/high) variants by editing the top of `adder_flow.tcl`. With only 1.14 ns of internal logic the path needs $0.4T + 1.14 \leq T - 0.08$, i.e. $T \geq 2.03$ ns, so the 3 ns run should pass with about +0.58 ns slack.

Note: data_out always shows register A, so it shows moving values during the addition. The result is valid when RDY is high.

7 What I learned

- Serial arithmetic trades time for hardware: 1 full adder instead of 8, but 9 clocks per addition.
- In a serial design the control logic, not the math, can be the slowest path.
- Changing testbench inputs on the falling edge makes RTL and gate-level results match exactly.
- A `.tcl` flow script makes synthesis repeatable and keeps every run's reports separate.

A Source files

RTL: `src/assignment2.sv`

```
// =====
// Datapath: registers, full adder, carry register, bit counter
// =====
```

```

`timescale 1ns/10ps
module datapath (
    input logic      clk,
    input logic      rst_n,
    input logic [7:0] data_in,
    output logic [7:0] data_out,
    input logic      ld_a,
    input logic      ld_b,
    input logic      shift_en,
    input logic      clr_carry,
    input logic      set_carry,
    output logic      carry_out,
    output logic      done_count
);

    logic [7:0] reg_a, reg_b;
    logic      carry_reg;
    logic [3:0] bit_count;
    logic      sum, carry_next;

    // Single 1-bit full adder (combinational)
    assign sum      = reg_a[0] ^ reg_b[0] ^ carry_reg;
    assign carry_next = (reg_a[0] & reg_b[0]) | (reg_a[0] & carry_reg) | (reg_b[0] & carry_reg);

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            reg_a      <= 8'b0;
            reg_b      <= 8'b0;
            carry_reg <= 1'b0;
            bit_count <= 4'b0;
        end else begin
            if (ld_a)      reg_a <= data_in;
            else if (shift_en) reg_a <= {sum, reg_a[7:1]};

            if (ld_b)      reg_b <= data_in;
            else if (shift_en) reg_b <= {1'b0, reg_b[7:1]};

            if (clr_carry)  carry_reg <= 1'b0;
            else if (set_carry) carry_reg <= 1'b1;
            else if (shift_en) carry_reg <= carry_next;

            if (ld_b)      bit_count <= 4'b0;
            else if (shift_en) bit_count <= bit_count + 1'b1;
        end
    end

    assign data_out  = reg_a;
    assign carry_out = carry_reg;
    assign done_count = (bit_count == 4'd8);

endmodule

// =====
// Control unit: 3-state FSM (IDLE -> ADD -> DONE -> IDLE)
// =====
module control_unit (
    input logic clk,
    input logic rst_n,
    input logic ldA,
    input logic ldB,
    input logic done_count,
    output logic ld_a,
    output logic ld_b,

```

```

output logic shift_en,
output logic rdy
);

typedef enum logic [1:0] {IDLE, ADD, DONE} state_t;
state_t state, next_state;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) state <= IDLE;
    else state <= next_state;
end

always_comb begin
    ld_a      = 1'b0;
    ld_b      = 1'b0;
    shift_en  = 1'b0;
    rdy       = 1'b0;
    next_state = state;

    case (state)
        IDLE: begin
            if (ldA) ld_a = 1'b1;
            if (ldB) begin
                ld_b      = 1'b1;
                next_state = ADD;
            end
        end

        ADD: begin
            if (!done_count) shift_en = 1'b1;
            if (done_count) next_state = DONE;
        end

        DONE: begin
            rdy      = 1'b1;
            next_state = IDLE;
        end

        default: next_state = IDLE;
    endcase
end

endmodule

// =====
// Top level: connects datapath + control unit, external interface
// =====
module serial_adder (
    input logic      clk,
    input logic      rst_n,
    input logic [7:0] data_in,
    output logic [7:0] data_out,
    input logic      ldA,
    input logic      ldB,
    input logic      clrCarry,
    input logic      setCarry,
    output logic      CarryOut,
    output logic      RDY
);

logic ld_a, ld_b, shift_en, done_count;

control_unit u_ctrl (
    .clk      (clk),

```

```

        .rst_n      (rst_n),
        .ldA        (ldA),
        .ldB        (ldB),
        .done_count (done_count),
        .ld_a        (ld_a),
        .ld_b        (ld_b),
        .shift_en    (shift_en),
        .rdy         (RDY)
    );

    datapath u_dp (
        .clk      (clk),
        .rst_n     (rst_n),
        .data_in   (data_in),
        .data_out  (data_out),
        .ld_a      (ld_a),
        .ld_b      (ld_b),
        .shift_en  (shift_en),
        .clr_carry (clrCarry),
        .set_carry (setCarry),
        .carry_out (CarryOut),
        .done_count (done_count)
    );

endmodule

```

Testbench: src/assignment2_tb.sv

```

`timescale 1ns/10ps

module assignment2_tb;

    logic      clk = 0;
    logic      rst_n;
    logic [7:0] data_in, data_out;
    logic      ldA, ldB, clrCarry, setCarry;
    logic      CarryOut, RDY;

    serial_adder dut (
        .clk      (clk),
        .rst_n     (rst_n),
        .data_in   (data_in),
        .data_out  (data_out),
        .ldA       (ldA),
        .ldB       (ldB),
        .clrCarry  (clrCarry),
        .setCarry  (setCarry),
        .CarryOut  (CarryOut),
        .RDY       (RDY)
    );

    always #5 clk = ~clk;    // 10ns clock period

    task automatic do_add(input [7:0] a, input [7:0] b);
        @(negedge clk);
        clrCarry = 1'b1;      // start each addition with carry cleared
        @(negedge clk);
        clrCarry = 1'b0;
        data_in = a; ldA = 1'b1;
        @(negedge clk);
        ldA = 1'b0;
        data_in = b; ldB = 1'b1;
    endtask

```

```

    @(negedge clk);
    ldB = 1'b0;
    wait (RDY);
    @(negedge clk);
    $display("A=%0d B=%0d -> result=%0d carry=%0d", a, b, data_out, CarryOut);
endtask

task automatic do_add_cin(input [7:0] a, input [7:0] b);
    @(negedge clk);
    setCarry = 1'b1;           // carry-in = 1
    @(negedge clk);
    setCarry = 1'b0;
    data_in = a; ldA = 1'b1;
    @(negedge clk);
    ldA = 1'b0;
    data_in = b; ldB = 1'b1;
    @(negedge clk);
    ldB = 1'b0;
    wait (RDY);
    @(negedge clk);
    $display("A=%0d B=%0d +cin -> result=%0d carry=%0d", a, b, data_out, CarryOut);
endtask

initial begin
    rst_n = 0; ldA = 0; ldB = 0; clrCarry = 0; setCarry = 0; data_in = 0;
    @(negedge clk);
    @(negedge clk);
    rst_n = 1;

    do_add(8'd3, 8'd5);        // expect result=8, carry=0
    do_add(8'd200, 8'd100);    // expect result=44, carry=1 (300 wraps in 8 bits)
    do_add(8'd255, 8'd1);      // expect result=0, carry=1
    do_add_cin(8'd3, 8'd5);    // expect result=9, carry=0
    do_add_cin(8'd255, 8'd0);  // expect result=0, carry=1

    #20 $finish;
end

endmodule

```

Synthesis script: src/adder_flow.tcl

```

#####
# adder_flow.tcl
# Lab 2, Assignment 2 -- 8-bit bit-serial adder (serial_adder)
# Design Compiler flow: analyze -> elaborate -> constrain ->
# compile -> reports -> netlist export -> Design Vision.
#
# Usage (from your assignment2/ directory):
#   dc_shell -f adder_flow.tcl
#
# Variants for the write-up (edit the four lines below, re-run):
#   baseline : CLK_PERIOD 5, MAP_EFFORT medium, AREA_EFFORT medium
#   speed    : CLK_PERIOD 3, MAP_EFFORT high, AREA_EFFORT low
#   area     : CLK_PERIOD 5, MAP_EFFORT medium, AREA_EFFORT high
# Report and netlist names get the variant name, so runs
# do not overwrite each other.
#####

# ----- EDIT THESE FOUR LINES PER VARIANT -----
set VARIANT      "baseline"
set CLK_PERIOD   5

```

```

set MAP_EFFORT      medium
set AREA_EFFORT     medium
# -----

# ---- library setup ----
set link_library [list \
  /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/synopsys/models/
  saed90nm_max.db \
  /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/synopsys/models/
  saed90nm_typ.db \
  /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/synopsys/models/
  saed90nm_min.db]

set target_library [list \
  /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/synopsys/models/
  saed90nm_typ.db]

# ---- TA note: never use BSLEX cells (they break tri-states) ----
# This design has no tri-state, so these lines are just a safety net.
set_dont_use [get_lib_cells */BSLEX1]
set_dont_use [get_lib_cells */BSLEX2]
set_dont_use [get_lib_cells */BSLEX4]

# ---- working library ----
define_design_lib WORK -path work

# ---- load design (top module = serial_adder) ----
analyze -library WORK -format sverilog {src/assignment2.sv}
elaborate serial_adder -architecture verilog -library WORK
current_design serial_adder
link

check_design > reports/adder_${VARIANT}_check_design.rpt

# ---- constraints ----
create_clock clk -name ideal_clock1 -period $CLK_PERIOD
set_input_delay [expr {$CLK_PERIOD * 0.4}] -clock ideal_clock1 \
  [remove_from_collection [all_inputs] clk]
set_output_delay [expr {$CLK_PERIOD * 0.4}] [all_outputs]
set_max_area 0

# ---- compile ----
compile -map_effort $MAP_EFFORT -area_effort $AREA_EFFORT

# ---- reports (auto-suffixed with the variant name) ----
report_area > reports/adder_${VARIANT}_area.rpt
report_timing > reports/adder_${VARIANT}_timing.rpt
report_qor > reports/adder_${VARIANT}_qor.rpt
report_resources > reports/adder_${VARIANT}_resources.rpt
report_constraints > reports/adder_${VARIANT}_constraints.rpt

# ---- netlist export (-hierarchy keeps datapath + control_unit) ----
write -format verilog -hierarchy -output output/adder_${VARIANT}_synth.v
write -format ddc -hierarchy -output output/adder_${VARIANT}.ddc
write_sdc constraints/adder_${VARIANT}.sdc

puts "\n== Variant '${VARIANT}' complete: period=${CLK_PERIOD}ns, map_effort=${MAP_EFFORT},
      area_effort=${AREA_EFFORT} ==="
puts "=== Reports written to reports/adder_${VARIANT}*.rpt ==="
puts "=== Netlist written to output/adder_${VARIANT}_synth.v ==="

# ---- show critical path on screen, then open Design Vision ----
report_timing
gui_start

```

```
# quit
```